Lecture 9: Arrays, Memory Management, and Basic Array Algorithms

1. Introduction to Arrays & Memory Allocation

When dealing with large amounts of related data, declaring individual variables (e.g., `int a, b, c...`) becomes computationally unscalable. Arrays solve this by acting as a single, consolidated data structure.

- **Definition:** An array is a data structure used to store a collection of items of the *same* data type at contiguous (adjacent) memory locations.
- **0-Based Indexing:** In C++, array indexing always begins at `0`. If an array has N elements, the final element is located at index $N-1$.
- **Memory Scaling:** If an array starts at a specific memory address (the base address, e.g., `200`), subsequent elements are found by adding the byte-size of the data type. For an `int` array (taking 4 bytes per element), indices will map to addresses like `200`, `204`, `208`, etc..
- **Initialization Tricks:** * Initializing entirely with zeros: `int arr[10] = {0};` perfectly fills all indices with 0.
  - Initializing with other numbers: `int arr[10] = {1};` places a `1` at the 0^{th} index, but automatically fills the rest of the array with 0 s.
- **Dynamic Size Calculation:** To programmatically find the number of elements in an array, divide the total memory size of the array by the size of a single element: `sizeof(arr) / sizeof(int)`.

2. The Array Scope Protocol: Pass-by-Reference

Unlike standard scalar variables (which are passed into functions as isolated copies), arrays operate on a fundamentally different set of rules in C++.

- **Base Address Transmission:** When you pass an array to a function (e.g., `update(arr, size)`), you are strictly passing the *memory address of the first element* (the pointer), not a duplicate copy of the array's contents.
- **In-Place Modification:** Because the function operates directly on the original memory addresses, any changes made to the array inside the function (e.g., `arr[0] = 120;`) will permanently alter the original array in the `main()` function. This behavior is known as **Pass-by-Reference**.

Problem-Solving Deconstruction

Problem 1: Find Maximum and Minimum in an Array

1. Problem Statement & Constraints: Given an array of N integers, iterate through the elements to mathematically extract the absolute maximum and absolute minimum values.

2. Core Intuition: We cannot safely initialize our tracking variables (`maxi` and `mini`) with 0, because the array might contain entirely negative or entirely positive numbers. Instead, we must initialize `mini` to the largest possible integer (`INT_MAX`) and `maxi` to the smallest possible integer (`INT_MIN`). This guarantees that the very first array element evaluated will instantly overwrite these trackers.

3. Algorithmic Steps: 1. Initialize `mini = INT_MAX` and `maxi = INT_MIN`.

2. Open a `for` loop executing from $i = 0$ up to $N - 1$.
3. For minimum: Overwrite `mini` with the result of the built-in `min(mini, num[i])` function.
4. For maximum: Overwrite `maxi` with the result of the built-in `max(maxi, num[i])` function.
5. Return or print the trackers.

4. Dry Run:

Input Array: `[4, 12, -2]`

1. `maxi = INT_MIN`. Check `4`: `maxi = max(INT_MIN, 4) = 4`.
2. `maxi = 4`. Check `12`: `maxi = max(4, 12) = 12`.
3. `maxi = 12`. Check `-2`: `maxi = max(12, -2) = 12`.

Final Maxi: `12`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The array is strictly traversed exactly once to check every element.
Space Complexity	$O(1)$	No additional arrays or dynamic memory allocations are used.

C++

```
#include<iostream>
using namespace std;

int getMin(int num[], int n) {
    int mini = INT_MAX;
    for(int i = 0; i < n; i++) {
        mini = min(mini, num[i]); // Built-in comparison
    }
    return mini;
```

```
}

int getMax(int num[], int n) {
    int maxi = INT_MIN;
    for(int i = 0; i < n; i++) {
        maxi = max(maxi, num[i]); // Built-in comparison
    }
    return maxi;
}

int main() {
    int size;
    cin >> size;
    int num[100];

    for(int i = 0; i < size; i++) {
        cin >> num[i];
    }

    cout << "Maximum value is " << getMax(num, size) << endl;
    cout << "Minimum value is " << getMin(num, size) << endl;
    return 0;
}
```

Problem 2: Sum of Array Elements

1. Problem Statement & Constraints: Given an array of size N, compute and return the total sum of all internal elements.

2. Core Intuition: Utilize the standard "accumulator" pattern. Initialize a variable at 0 and sequentially add the value of every array index during a single linear traversal.

3. Algorithmic Steps: 1. Read the array size N and elements into `arr[size]`.

2. Call `sumOfArray(arr, size)`.
3. Inside the function, initialize an accumulator `int sum = 0;`.
4. Run a `for` loop from $i = 0$ to $i < sie$.
5. Execute `sum += arr[i]`.
6. Return `sum`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	Iterates sequentially through all N elements exactly once.
Space Complexity	$O(1)$	Memory overhead is restricted to a single scalar `sum` variable.

Problem 3: Linear Search

1. Problem Statement & Constraints: Given an array of integers and a specific `key`, determine if the `key` exists anywhere within the array. Return a boolean True/False.

2. Core Intuition: Because the array is completely unsorted, we have no mathematical way to predict where an element might be. We must evaluate every single element one by one from left to right. If a match is found, we short-circuit (terminate) the algorithm immediately to save time.

3. Algorithmic Steps: 1. Define function `bool search(int arr[], int size, int key)`.

2. Iterate `for(int i = 0; i < size; i++)`.
3. Evaluate: `if (arr[i] == key)`.
4. If a match occurs, immediately `return 1;` (True).
5. If the entire loop finishes without triggering the `return 1`, the key does not exist. Execute a final `return 0;` (False).

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	In the worst-case scenario (key doesn't exist or is at the very end), N elements are checked.
Space Complexity	$O(1)$	Requires zero additional auxiliary space.

Problem 4: Reverse an Array

1. Problem Statement & Constraints: In-place reversal of an array. Do not create a second array; dynamically flip the elements within the existing memory structure.

2. Core Intuition: This requires the **Two-Pointer Paradigm**. We establish a pointer at the very front of the array and another pointer at the absolute rear. By repeatedly swapping the elements at these two pointers and physically moving the pointers toward the center, the array reverses itself automatically. The algorithm stops the moment the pointers cross or collide.

3. Algorithmic Steps: 1. Define pointers: `int start = 0;` and `int end = n - 1;`.

2. Open a `while` loop bound strictly by `start <= end`.
3. Swap the target elements: `swap(arr[start], arr[end]);`.
4. Advance the left pointer inward: `start++;`.

5. Retract the right pointer inward: `end--;`.
6. The loop naturally terminates when `start` bypasses `end`, signaling complete reversal.

4. Dry Run:

Input Array: `[1, 4, 0, 5]`

1. `start = 0` (val `1`), `end = 3` (val `5`). `0 <= 3` (True). Swap. Array: `[5, 4, 0, 1]`.
2. `start = 1` (val `4`), `end = 2` (val `0`). `1 <= 2` (True). Swap. Array: `[5, 0, 4, 1]`.
3. `start = 2`, `end = 1`. `2 <= 1` (False). Loop terminates. Reversal complete.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The two pointers meet in the middle, evaluating strictly $N/2$ swaps, simplifying asymptotically to $O(N)$.
Space Complexity	$O(1)$	Strictly an "In-Place" algorithm. Variables `start` and `end` take constant space.

C++

```
#include<iostream>
using namespace std;

void reverse(int arr[], int n) {
    int start = 0;
    int end = n - 1;

    // Two-Pointer execution loop
    while(start <= end) {
        swap(arr[start], arr[end]); // Built-in C++ swap mechanism
        start++;
        end--;
    }
}

void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
```

```
    int arr[6] = {1, 4, 0, 5, -2, 15};

    reverse(arr, 6);
    printArray(arr, 6);

    return 0;
}
```